

Chap.9 GAN (Generative Adversarial Network)

방 수 식 교수
(bang@tukorea.ac.kr)

한국공학대학교 전자공학부

2024년도 2학기
고급머신러닝실습 & 인공지능설계실습2

GAN(Generative Adversarial Network)



▪ 적대적 생성 신경망

- 앞에서 배운 VAE와 같은 생성 모델의 한 종류
- 2014년 **Ian Goodfellow**에 의해 제안
- 두 개의 네트워크 구조가 서로 **경쟁하는 과정**을 통해서 데이터를 학습
- 이미지 생성, 영상 생성, 텍스트 생성 등에 다양하게 응용



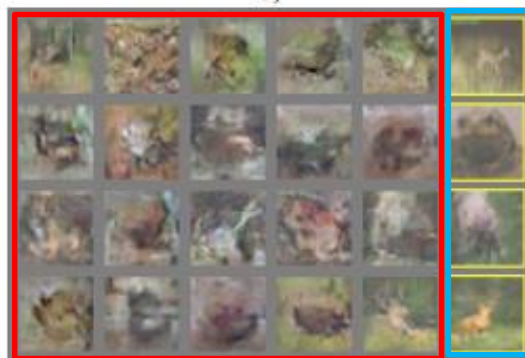
a)



b)



c)



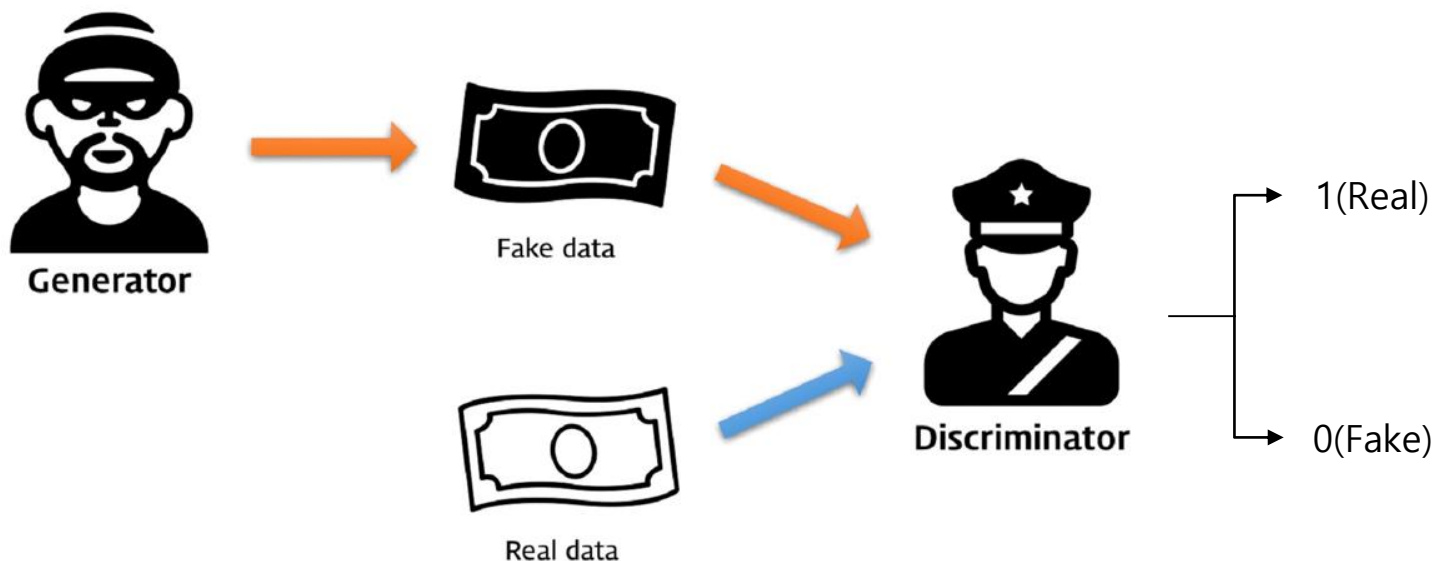
d)

 : 생성 이미지

 : 실제 이미지

· 네트워크 구조

- 새로운 이미지를 생성하는 생성자(Generator)
- 생성자가 생성한 이미지를 구분하는 판별자(Discriminator)
- 생성자(generator)와 판별자(discriminator)가 **적대적 관계**에서 경쟁하면서 훈련하는 방식
- 위조범은 **경찰을 속이기 위해**, 경찰은 실제 현금과 만들어진 위조지폐를 **구분 하기 위해 노력**

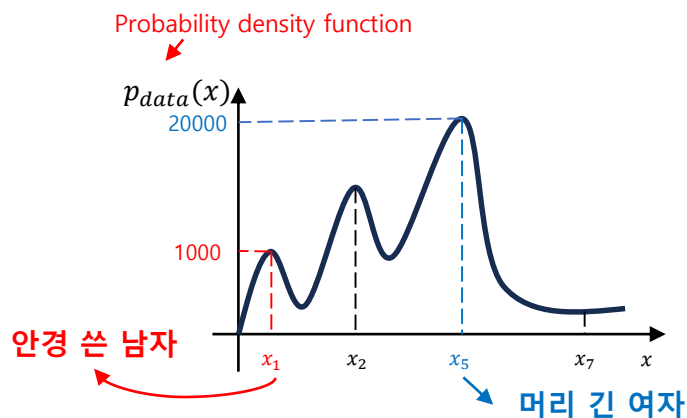


[참고] Probability Distribution

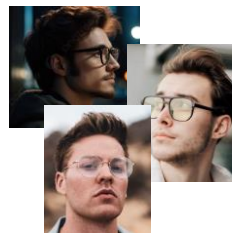


· 확률 분포 (1D 예시)

- 학습 데이터가 사람 얼굴 사진이라고 가정
- 학습 데이터에서 특징을 표현하는 값의 확률밀도값: x_n (ex. 안경 쓴 남자)



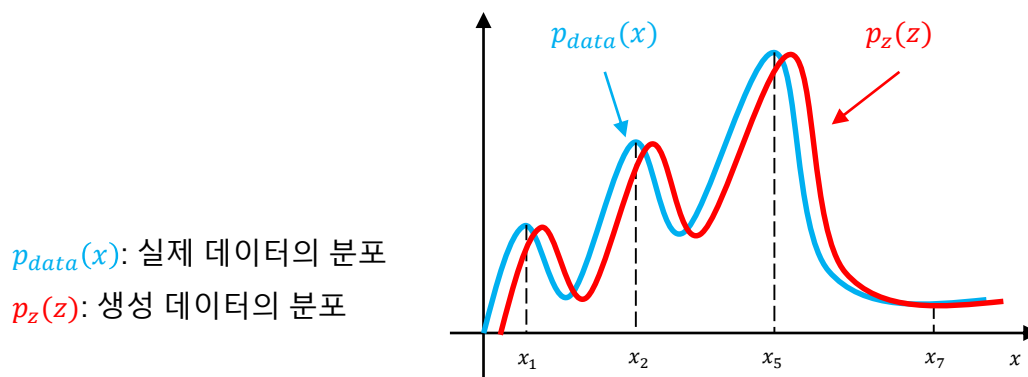
안경 쓴 남자
(1000 / 60000)



머리 긴 여자
(20000 / 60000)



- 생성자의 목표: 실제 데이터의 확률 분포에 근사하는 모델을 설계



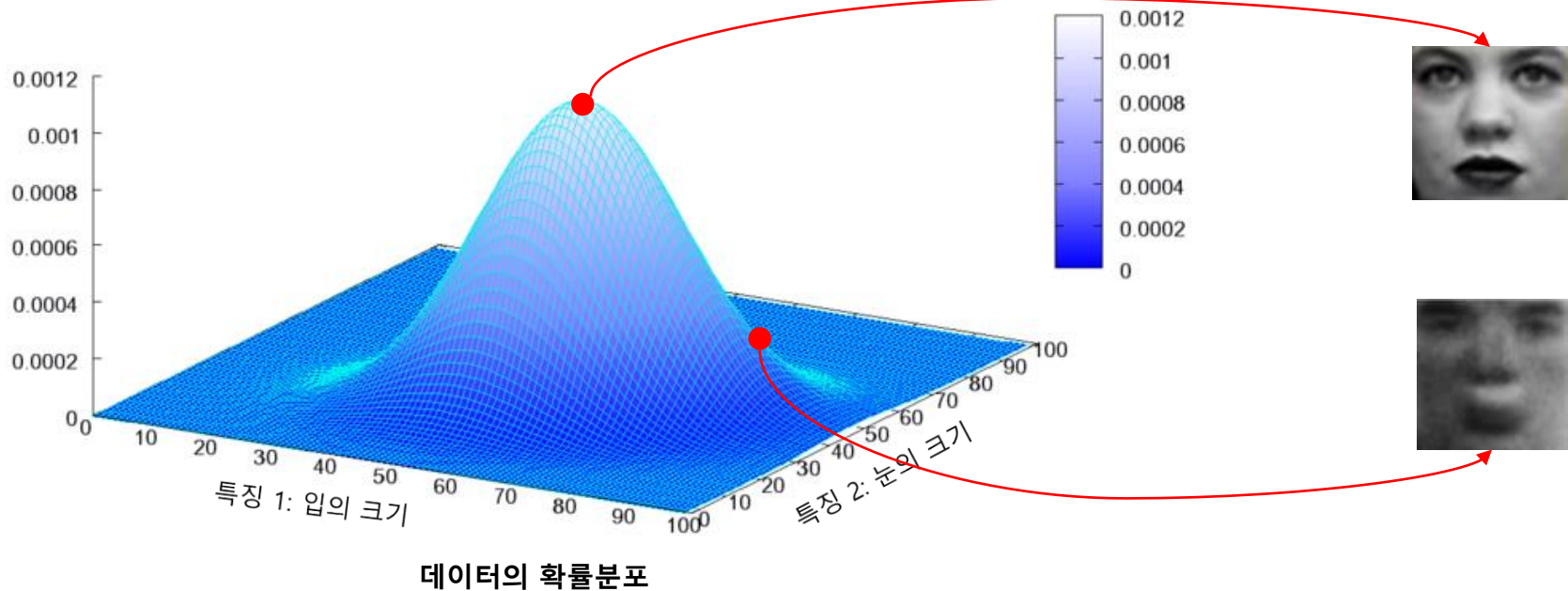
확률 변수의 차원을 늘려서
이미지 차원을 학습한다면?

[참고] Probability Distribution



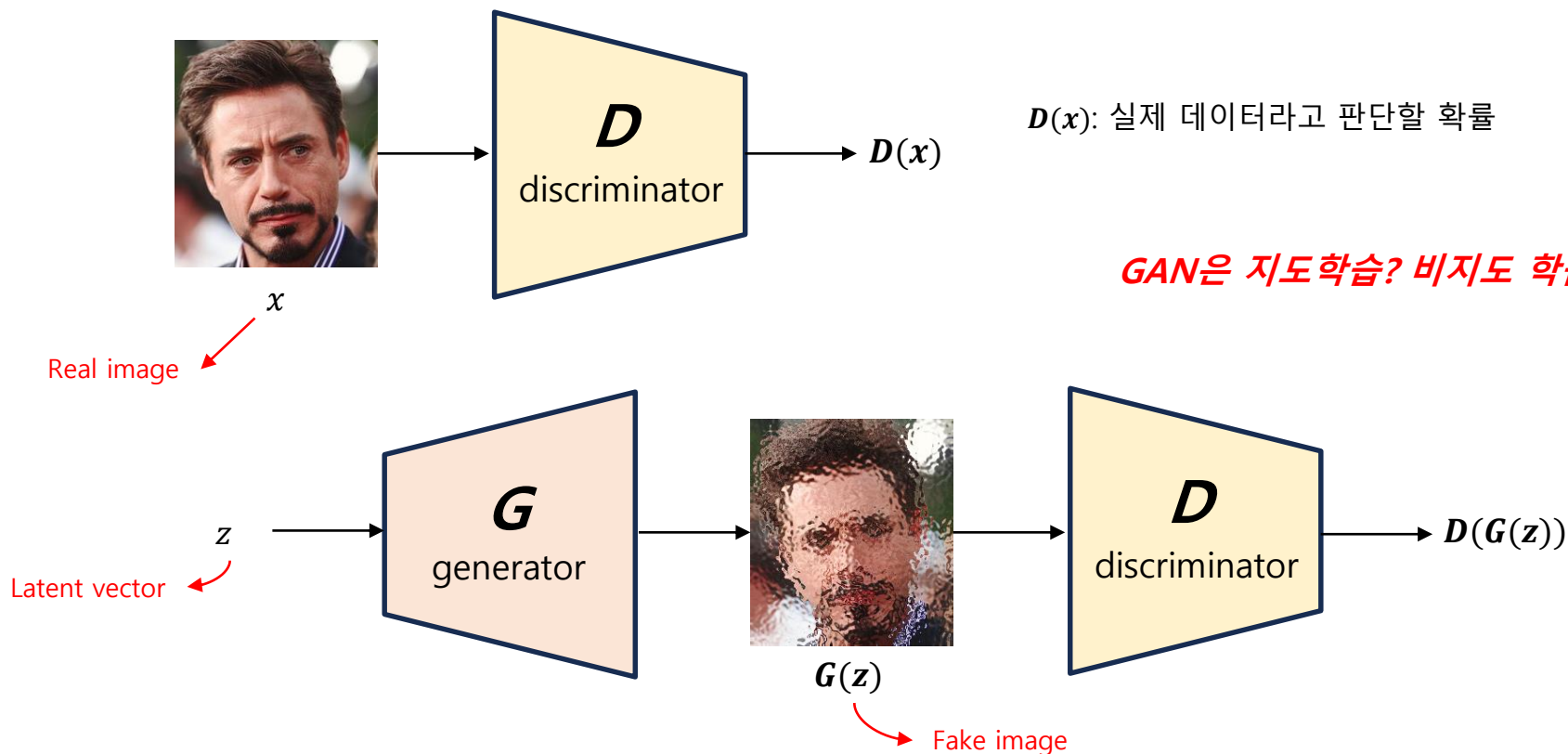
· 이미지에서의 확률 분포

- 이미지 데이터는 다차원 특징 공간의 한 점으로 표현가능
- ex) 두 개의 특징을 가지고 2차원에서 표현
- 실제 데이터의 분포와 생성 데이터의 분포 간의 차이를 감소하기 위해 모델을 설계



· 생성자와 판별자

- 생성자(Generator)는 최대한 진짜 같은 데이터를 만들기 위한 학습 진행
- 판별자(Discriminator)는 진짜와 가짜를 판별하기 위한 학습 진행
- 생성자와 판별자가 **적대적 관계**에서 **경쟁**하면서 학습

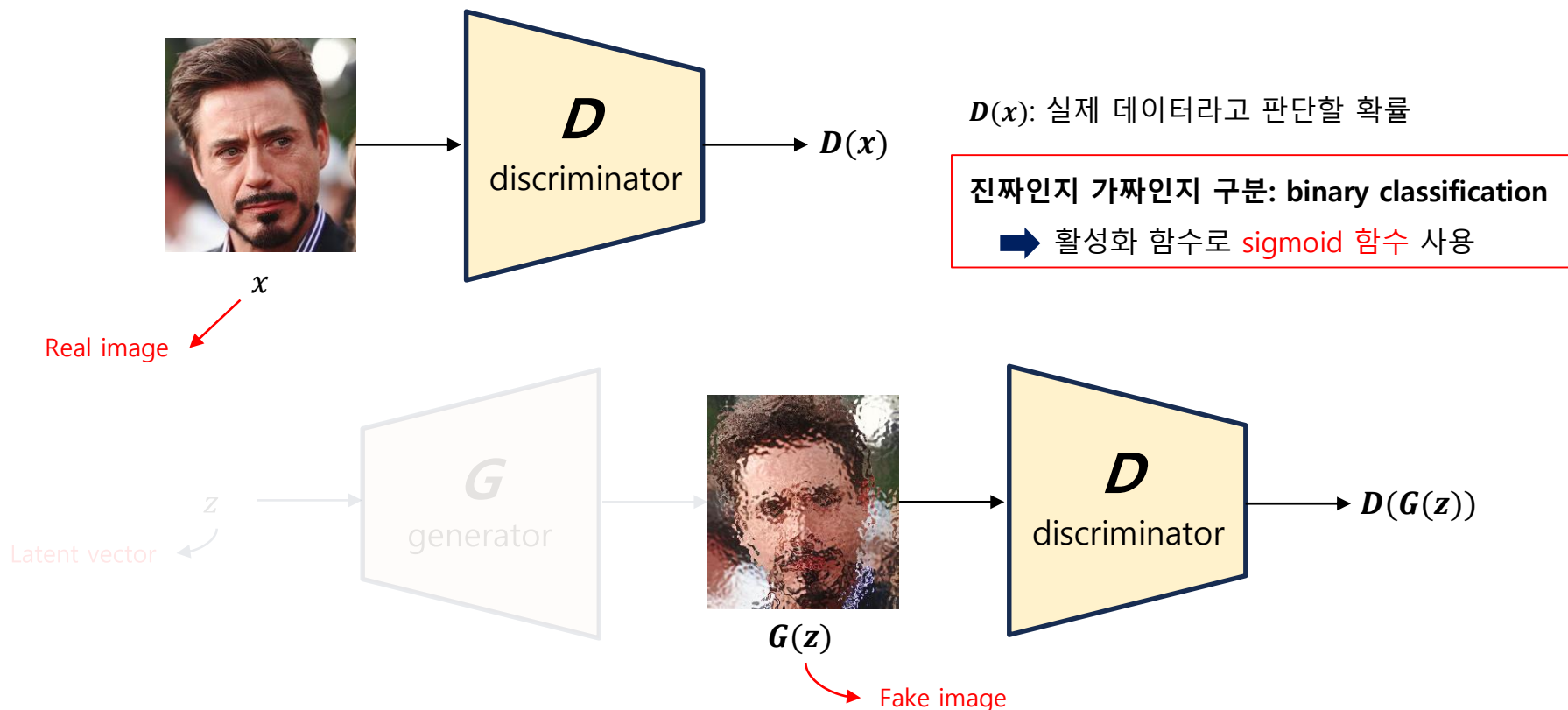


생성자와 판별자



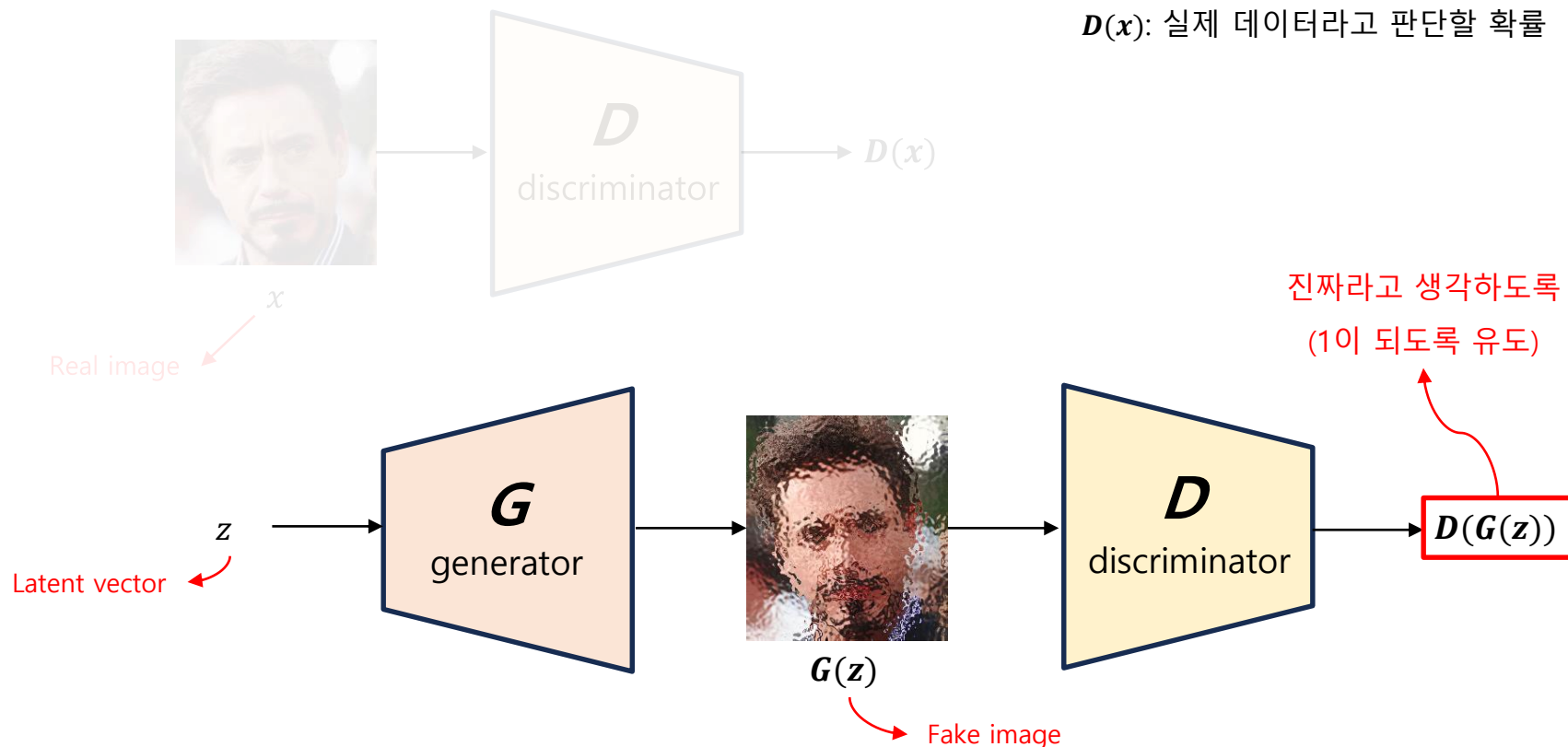
▪ 판별자(Discriminator)

- 판별자는 출력 label 있는 지도 학습
- 판별자는 실제 데이터가 들어갔을 때 진짜로 구분하도록 학습, 가짜 데이터가 들어갔을 때 가짜로 구분하도록 학습



· 생성자(Generator)

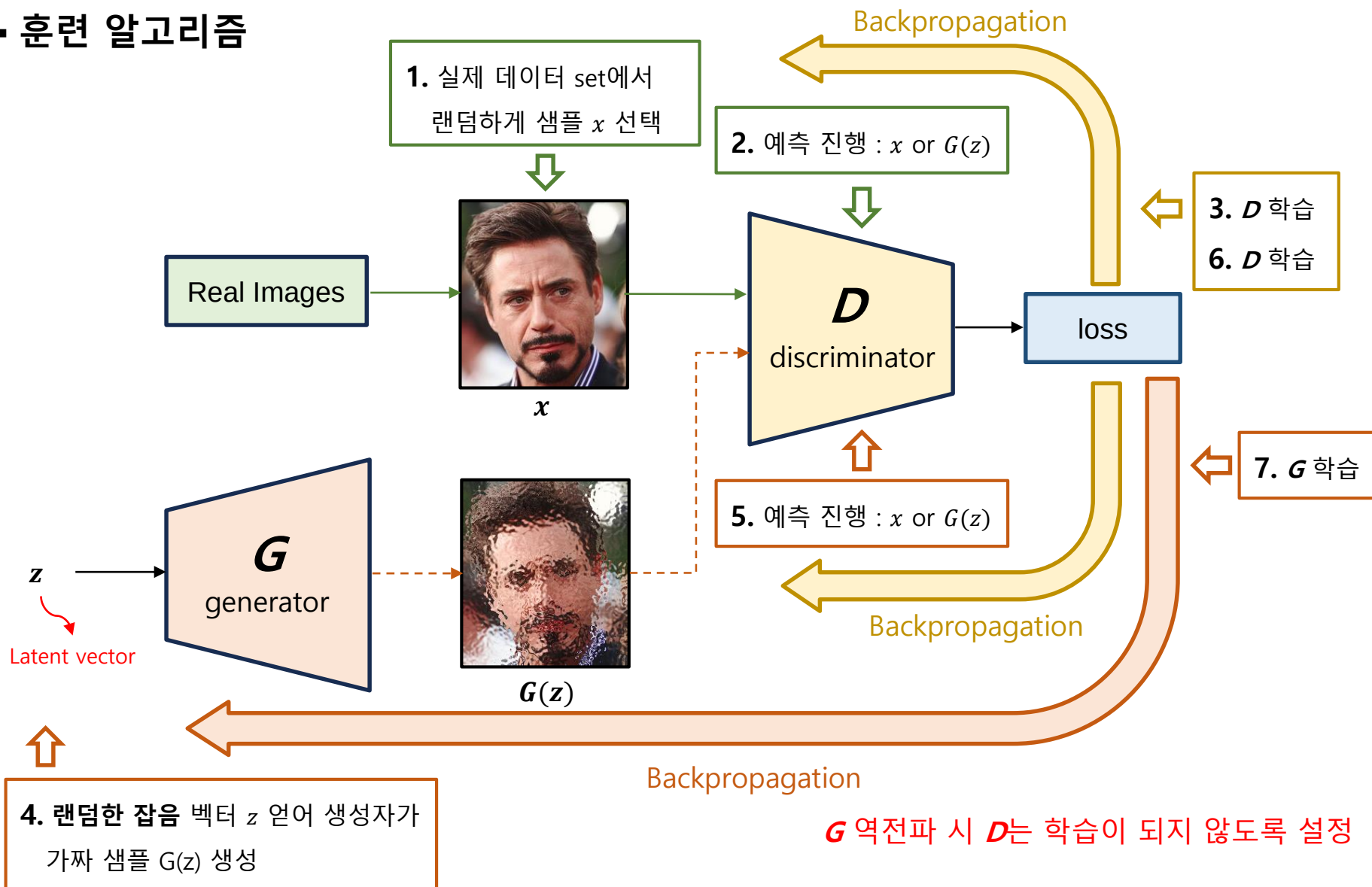
- 생성자의 경우 랜덤한 노이즈를 받아서 데이터를 생성, 데이터를 가지고 판별자를 속여 판별자가 실제데이터라고 구분하도록 유도, $D(G(z)) = 1$ 이 나오도록
- 학습하면 할수록 진짜 같은 가짜 데이터를 생성



GAN 훈련 알고리즘



· 훈련 알고리즘



목적 함수(Objective Function)



▪ 목적 함수

- 생성자는 목적함수를 최소화하는 방향으로, 판별자는 최대화하는 방향으로 학습
- 판별자와 생성자가 서로 번갈아가면서 학습

$$\min_G \max_D V(D, G) = E_{x \sim p_{data}(x)} [\log D(x)] + E_{z \sim p_z(z)} [\log(1 - D(G(z)))]$$

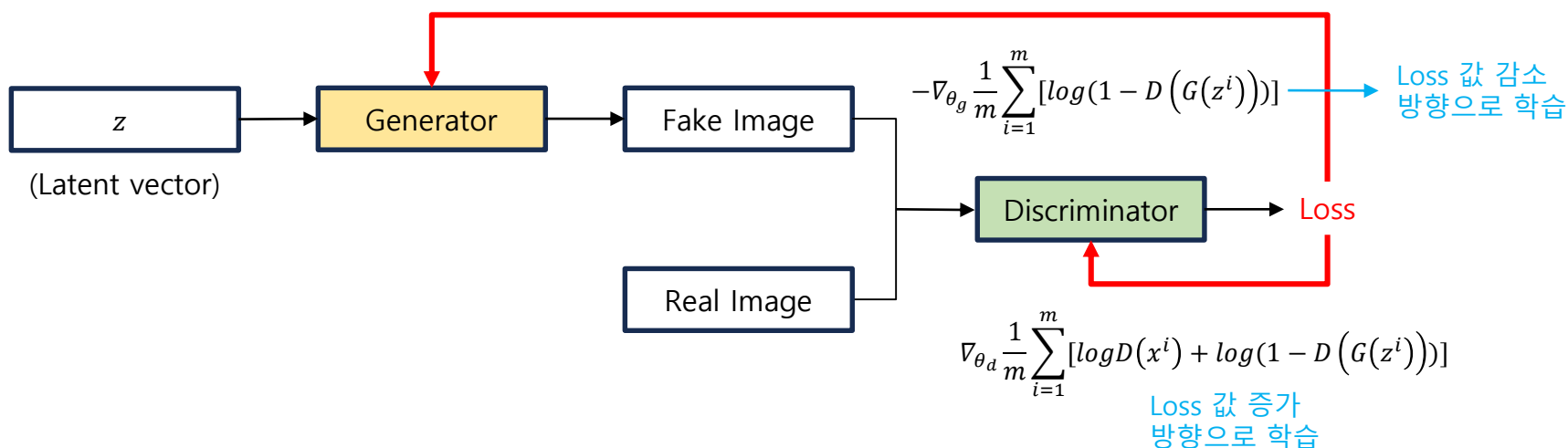
x : 실제 데이터의 샘플 이미지
 z : Latent vector
(일반적으로 가우스 분포)

Generator

$G(z)$: 생성 데이터 sample

Discriminator

$D(x)$: 실제 데이터일 확률(Real :1 ~ Fake:0)



목적 함수(Objective Function)



• 목적 함수(판별자 관점)

- 판별자는 목적함수를 **최대화**하는 방향으로 학습

x : 실제 데이터의 샘플 이미지
 z : Latent vector
(일반적으로 가우스 분포)

$G(z)$: 생성 데이터 sample

$D(x)$: 실제 데이터의 분포일 확률 (Real :1~ Fake:0)

진짜 이미지에 대한
확률 값이 1일 때
 $\log 1$ 로 최대화

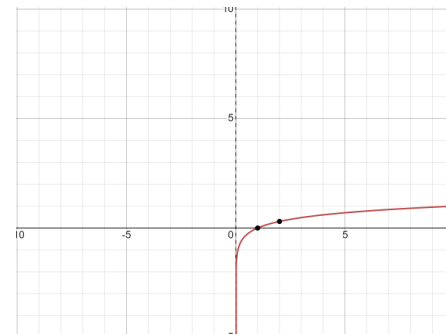
$$\min_G \max_D V(D, G) = E_{x \sim p_{data}(x)} [\log D(x)] + E_{z \sim p_z(z)} [\log(1 - D(G(z)))]$$

실제 데이터의 확률분포

Noise z 의 확률분포

가짜 이미지에 대한
확률 값이 0일 때
 $\log 1$ 로 최대화

- 판별자는 $V(D, G)$ 의 값을 높이하고자 실제 데이터에 대해서는 1로 분류할 수 있도록, 가짜 이미지에 대해서는 0으로 분류할 수 있도록 학습



<log 함수>

목적 함수(Objective Function)



▪ 목적 함수(생성자 관점)

- 생성자는 목적함수를 최소화하는 방향으로 학습

$G(z)$: 생성 데이터 sample

$D(x)$: 실제 데이터의 분포일 확률 (Real :1~ Fake:0)

x : 실제 데이터의 샘플 이미지

z : Latent vector

(일반적으로 가우스 분포)

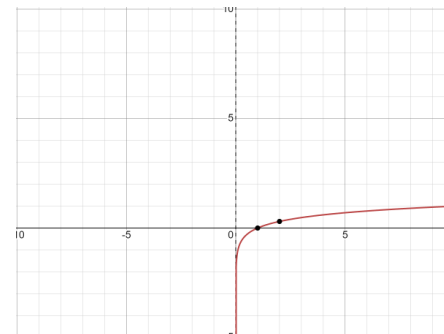
$$\min_G \max_D V(D, G) = E_{x \sim p_{data}(x)} [\log D(x)] + E_{z \sim p_z(z)} [\log(1 - D(G(z)))]$$

G 가 포함되지 않아 상수 취급

➡ 학습하지 않음

가짜 이미지에 대한
확률 값이 1일 때
 $\log 0$ 으로 최소화

- 생성자는 $V(D, G)$ 의 값을 낮추고자 자기가 만든 가짜 데이터를 생성자가 1로 분류할 수 있도록 학습



<log 함수>

목적 함수(Objective Function)



지능형 예측·진단 연구실
Intelligent Prognostics & Diagnostics Lab.

▪ 목적 함수(생성자 관점)

x : 실제 데이터의 샘플 이미지

z : Latent vector

(일반적으로 가우스 분포)

$G(z)$: 생성 데이터 sample

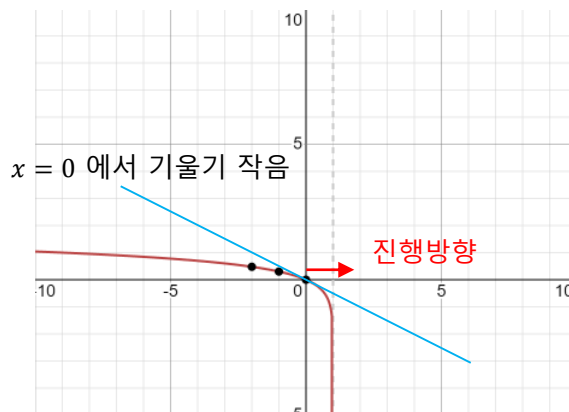
$D(x)$: 실제 데이터의 분포일 확률 (Real :1~ Fake:0)

$$\min_G \max_D V(D, G) = E_{x \sim p_{data}(x)} [\log D(x)] + E_{z \sim p_z(z)} [\log(1 - D(G(z)))]$$

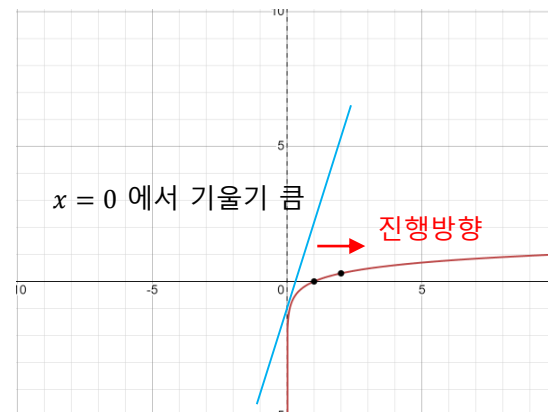
최적화가 더 용이



$$\max_G V(G) = E_{z \sim p_z(z)} [\log D(G(z))]$$



$\min \log(1-x)$
기울기가 완만하여 업데이트가 잘 안됨

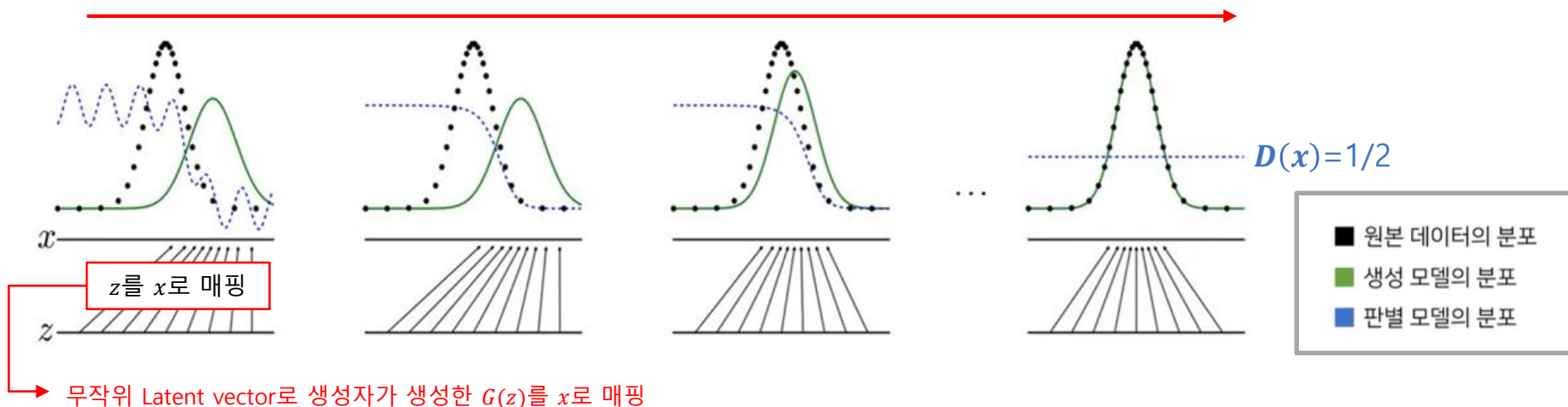


$\max \log(x)$
기울기가 급격하여 업데이트가 잘 됨

· 수렴 과정

- GAN 목표: 생성자의 분포가 실제 데이터의 분포를 잘 따를 수 있도록 설계
 - 즉, $P_g \rightarrow P_{data}$ 수렴
 - 학습이 전부 이루어지고 난 후 가짜 데이터와 진짜 데이터를 더 이상 구분 x
- ➡ $D(G(z)) \rightarrow 1/2$ 생성자가 내보는 가짜 데이터에 대해서 판별자는 구분할 수 없다.

학습 진행



- 초기 단계에는 생성자의 분포가 실제데이터의 분포를 학습하지 못해서 판별자가 잘 구분
- 학습이 진행되면서 생성자의 분포가 실제데이터의 분포를 적절히 따라감
- 학습이 완료되면 판별 모델의 분포는 **1/2로 수렴**

[참고] Jensen-Shannon Divergence

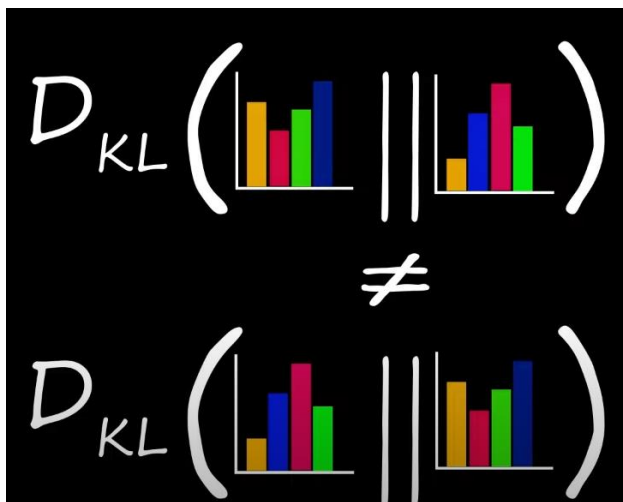


▪ Jensen-Shannon Divergence

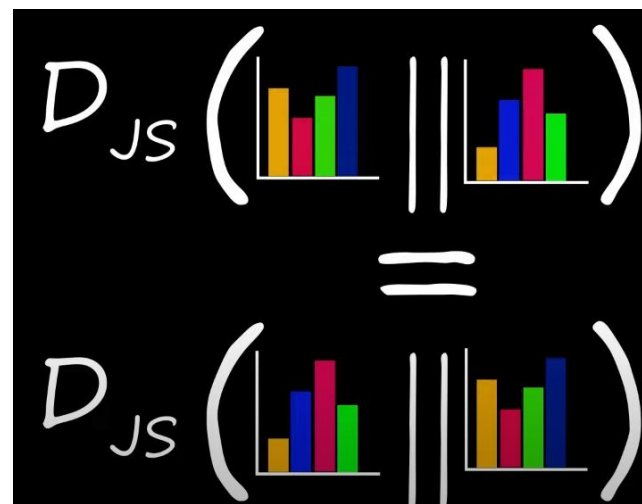
- 두 확률 분포 간의 유사성 측정
- KL Divergence는 **Symmetric(대칭)하지 않음** ➡ $D_{KL}(P||Q) \neq D_{KL}(Q||P)$
- JSD는 분포와 그 반대 분포 사이의 **KL Divergence 발산의 평균**을 취하는 대칭 버전
- JSD는 KL Divergence를 기반으로 하여 대칭적
- 두 확률 분포 사이의 Distance, 두 확률 분포가 동일할 때 **최솟값 0**을 가짐

$$JSD(P, Q) = \frac{1}{2}D_{KL}(P||M) + \frac{1}{2}D_{KL}(Q||M)$$

$$\because M = \frac{1}{2}(P + Q)$$



<Kullback Leibler Divergence>



<Jensen-Shannon Divergence>

[참고] 목적 함수의 수학적 증명



· 판별자 수학적 증명

- Proposition: $D^*(x) = \frac{p_{data}(x)}{p_{data}(x) + p_g(x)}$

- $p_g \rightarrow p_{data}$

- G가 고정됐다고 가정

$$D^*(x) = \arg \max_D V(G, D) = E_{x \sim p_{data}(x)} [\log D(x)] + E_{z \sim p_z(z)} [\log (1 - D(G(z)))]$$

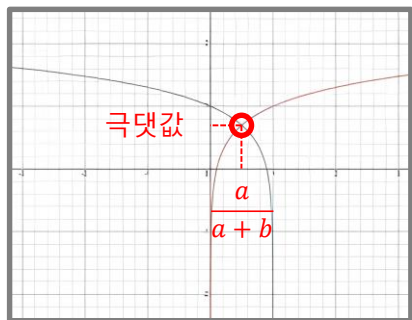
$$= \int_x p_{data}(x) \log(D(x)) dx + \int_z p_z(z) \log(1 - D(g(z))) dz$$

$E[x] = \int_{-\infty}^{\infty} xf(x)dx$

z가 x로 매핑되는 과정

$$= \int_x \{p_{data}(x) \log(D(x)) + p_g(x) \log(1 - D(x))\} dx$$

function $y \rightarrow a \log(y) + b \log(1 - y)$ y값은 $\frac{a}{a+b}$ 에서 극댓값을 가짐



$$\therefore D^*(x) = \frac{p_{data}(x)}{p_{data}(x) + p_g(x)} \text{ 성립}$$

$$a \log(y) + b \log(1 - y)$$

$$\frac{a}{y} + \frac{-b}{1-y} = \frac{a - (a+b)y}{y(1-y)} = 0$$

y로 미분
0되는 지점

$$\therefore y = \frac{a}{a+b}$$

[참고] 목적 함수의 수학적 증명



· 생성자 수학적 증명

- *Proposition:* 생성자의 Global optimum point ($p_g = p_{data}$)

$$\min_G \max_D V(D, G) = \min_G V(D^*, G)$$

$$V(D^*, G) = E_{x \sim p_{data}(x)} [\log(D^*(x))] + E_{z \sim p_z(z)} [\log(1 - D^*(G(z)))]$$

$$D^*(x) = \frac{p_{data}(x)}{p_{data}(x) + p_g(x)}$$

$$= \int_x p_{data}(x) \left[\log \frac{p_{data}(x)}{p_{data}(x) + p_g(x)} \right] dx + \int_x p_g(x) \left[\log \frac{p_g(x)}{p_{data}(x) + p_g(x)} \right] dx$$

$$= -\log 4 + \log 4 + \int_x p_{data}(x) \left[\log \frac{p_{data}(x)}{p_{data}(x) + p_g(x)} \right] dx + \int_x p_g(x) \left[\log \frac{p_g(x)}{p_{data}(x) + p_g(x)} \right] dx$$

$$D_{KL}(B||A) = E_{x \sim B} \left[\log \frac{B(x)}{A(x)} \right] = \sum_x B(x) \log \frac{B(x)}{A(x)} = \int_{-\infty}^{\infty} B(x) \log \left(\frac{B(x)}{A(x)} \right) dx$$

$$= -\log 4 + \int_x p_{data}(x) \left[\log \frac{2 * p_{data}(x)}{p_{data}(x) + p_g(x)} \right] dx + \int_x p_g(x) \left[\log \frac{2 * p_g(x)}{p_{data}(x) + p_g(x)} \right] dx$$

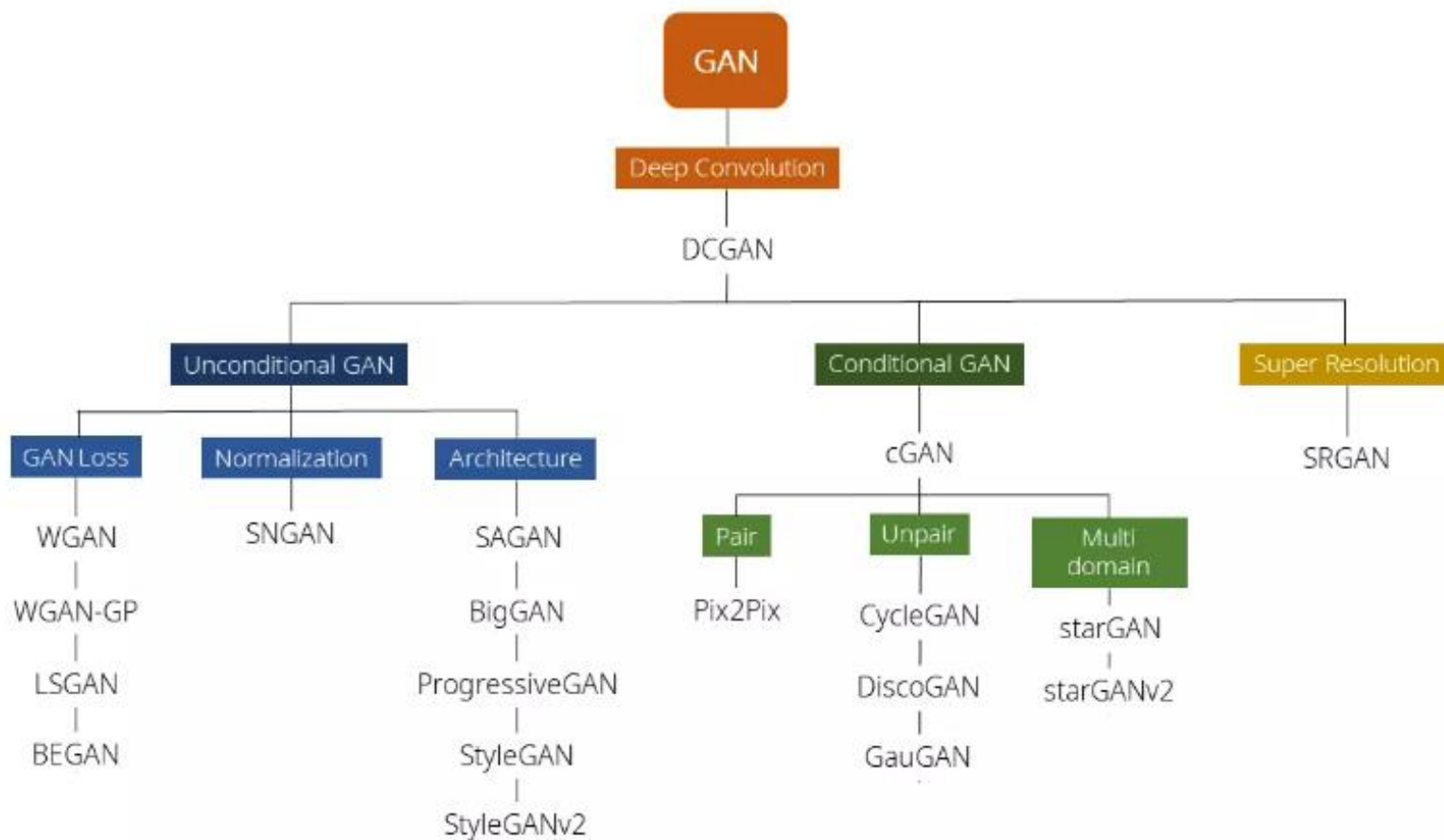
$$= KL(p_{data} || \frac{p_{data}(x) + p_g(x)}{2}) + KL(p_g || \frac{p_{data}(x) + p_g(x)}{2}) - \log(4)$$

$$= 2 * JSD(p_{data} || p_g) - \log(4)$$

$$JSD(p||q) = \frac{1}{2} D_{KL}(p || \frac{p+q}{2}) + \frac{1}{2} D_{KL}(q || \frac{p+q}{2})$$

$p_g = p_{data}$ 일때 JSD는 최솟값 0을 가짐 ➡ $D^*(x) = \frac{p_{data}(x)}{p_{data}(x) + p_g(x)} = \frac{1}{2}$

· GAN 활용 모델

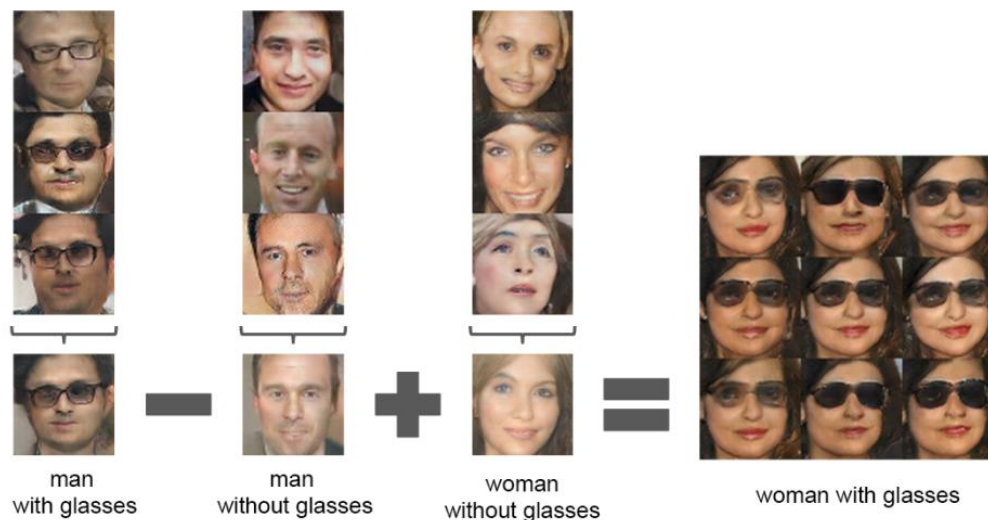
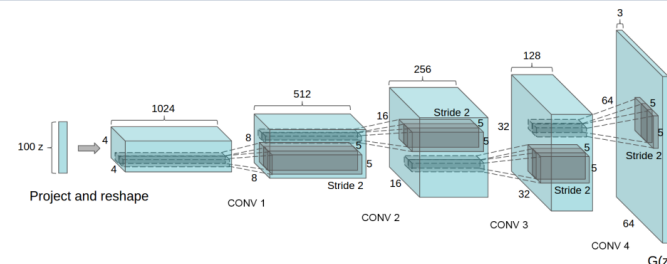


GAN의 고도화 1



▪ DCGAN(Deep Convolutional GAN)

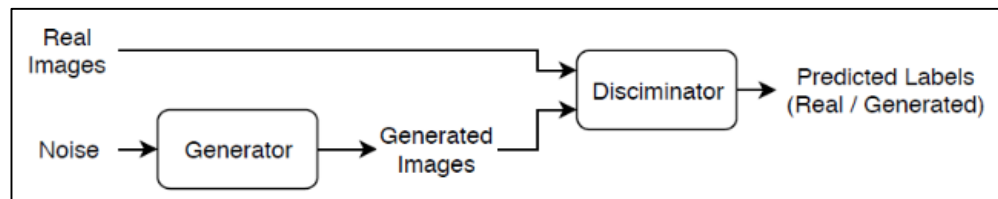
- GAN 발표 이후 불안정한 구조로 인한 문제 발생
- GAN에 **Convolutional layer** 적용하여 안정화
- Generator: **Transposed Convolutional layer** 활용
- Discriminator: **Convolutional layer** 활용
- 대부분의 상황에서 언제나 안정적으로 학습이 되는 Convolutional layer 적용
- z-space의 연산이 가능



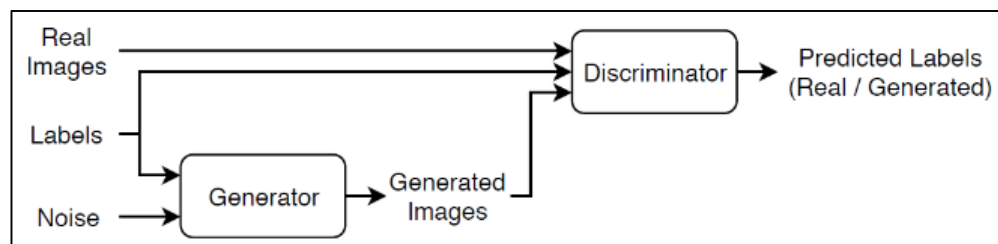
ex. "안경 쓴 남자"에 해당하는 z-vector에서 "안경을 쓰지 않은 남자"에 해당하는 z-vector를 빼고, "안경을 쓰지 않은 여자"에 해당하는 z-vector를 더하면 그 z-vector로 생성한 이미지는 "안경을 쓴 여자"가 됨

▪ cGAN(Conditional GAN)

- Generator에 “내가 원하는 방향성”을 부여하는 방법
- Discriminator와 Generator에 Label 벡터를 함께 입력시킴



일반적 GAN



cGAN

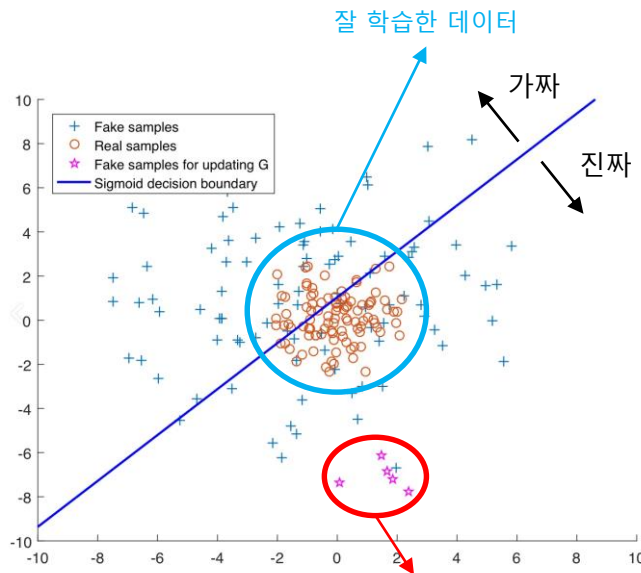
$$\min_G \max_D V(D, G) = \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}(\mathbf{x})} [\log D(\mathbf{x}|\mathbf{y})] + \mathbb{E}_{\mathbf{z} \sim p_z(\mathbf{z})} [\log(1 - D(G(\mathbf{z}|\mathbf{y})))]$$

GAN의 고도화 3

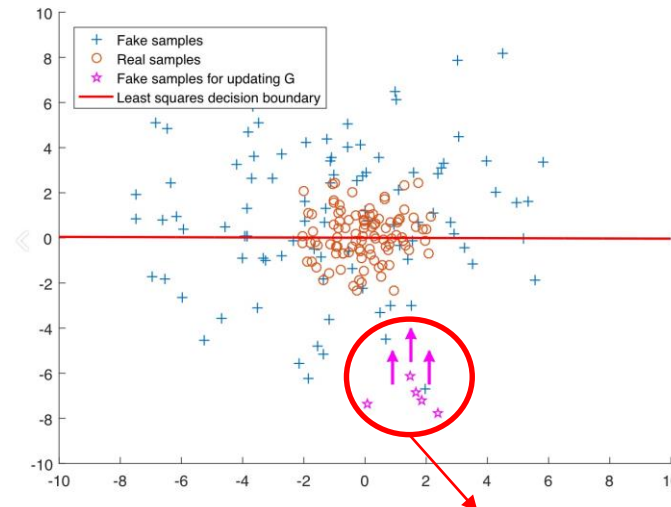


▪ LSGAN(Least Squares GAN)

- 기존의 GAN 모델에서 발생하는 **Gradient Vanishing 문제**를 해결
- Discriminator를 속이는 임무를 완수한 것으로 간주, 더 이상 **Generator 업데이트에 도움 안 됨**
- Decision boundary에서 멀리 떨어진 sample에게 **penalty**를 주어 진짜 데이터에 근접



Discriminator를 완벽하게 속임
-> 좋지 않음



데이터를 decision boundary쪽으로 가져와
실제데이터와 비슷하게 만들

▪ MNIST 생성

- MNIST 데이터셋 취득

```
mnist = keras.datasets.mnist
(x_train, y_train), (x_test, y_test) = mnist.load_data()

x_train = x_train / 127.5 - 1
x_test = x_test / 127.5 - 1

x_train = x_train.reshape(len(x_train), 784)
x_test = x_test.reshape(len(x_test), 784)
```

Generator에서 활성화 함수로 tanh 사용
➡ 범위를 -1~1사이로 설정

- Generator 설계

```
noise_dimension = 10 #noise 차원
batch_size = 64
iteration = 1000

generator = keras.models.Sequential([
    keras.layers.Dense(256, activation = 'relu', input_dim = noise_dimension),
    keras.layers.Dense(512, activation = 'relu'),
    keras.layers.Dense(1024, activation = 'relu'),
    keras.layers.Dense(28*28, activation='tanh'),
])

generator.summary()
```

Layer (type)	Output Shape	Param #
dense_296 (Dense)	(None, 256)	2816
dense_297 (Dense)	(None, 512)	131584
dense_298 (Dense)	(None, 1024)	525312
dense_299 (Dense)	(None, 784)	803600
Total params: 1,463,312		
Trainable params: 1,463,312		
Non-trainable params: 0		

모델 구조 확인

▪ MNIST 생성

- Discriminator 설계

```
discriminator = keras.models.Sequential([
    keras.layers.Dense(1024, activation = 'relu', input_shape = (784,)),
    keras.layers.Dropout(0.3),
    keras.layers.Dense(512, activation = 'relu'),
    keras.layers.Dropout(0.3),
    keras.layers.Dense(256, activation = 'relu'),
    keras.layers.Dropout(0.3),
    keras.layers.Dense(1, activation='sigmoid'),
])

discriminator.summary()
```

- Generator, Discriminator 연결 (Functional API)

```
discriminator.compile(loss = 'binary_crossentropy', optimizer = 'adam')
discriminator.trainable = False #discriminator 고정
gan_input = keras.layers.Input(shape=(noise_dimension,)) #10차원
fake_image = generator(inputs=gan_input) #가짜 이미지 생성
gan_output = discriminator(fake_image) #가짜인지 진짜인지 판별

gan = keras.Model(gan_input, gan_output)
gan.compile(loss='binary_crossentropy', optimizer = 'adam')
```

Discriminator는 학습되지 않도록
설정 후 시작

→ 왜 그냥 cross entropy를 사용할까?

▪ MNIST 생성

- Generator, Discriminator 학습

```
def train_discriminator(x_train): #Discriminator 학습 함수
    discriminator.trainable = True #D 학습 가능하도록 설정
    c= np.random.randint(0, x_train.shape[0], batch_size) #학습할 batch 설정
    real = x_train[c] #이번 iteration에 사용할 이미지 batch
    #discriminator 학습 => 실제 이미지
    discriminator.train_on_batch(real, np.ones((batch_size,1)))

    p=np.random.normal(0,1, (batch_size,noise_dimension)) #노이즈
    fake = generator.predict(p) #가짜 이미지 생성
    #discriminator 학습 => 가짜 이미지
    discriminator.train_on_batch(fake, np.zeros((batch_size,1)))

def train_generator(): #Generator 학습 함수
    discriminator.trainable = False #D를 다시 학습 불가능 하도록
    p=np.random.normal(0,1, (batch_size,noise_dimension)) #노이즈
    #generator 학습
    gan.train_on_batch(p, np.ones((batch_size,1)))
    """ generator 학습 과정
    1. discriminator를 학습 불가능 하도록 설정
    2. generator와 discriminator가 함께 있는 gan 모델 자체 학습
    --> generator만 학습됨
    """
```

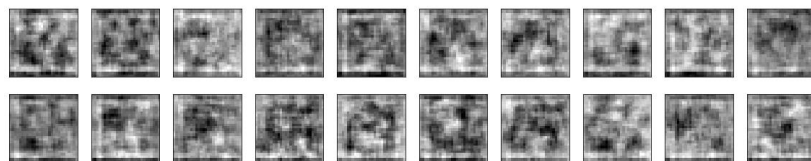
GAN 실습



▪ MNIST 생성

- 생성 이미지 시각화

```
for i in range(iteration+1): # 학습을 수행
    train_discriminator(x_train)
    train_generator()
    if(i%100==0): # 학습 도중 100 epoch마다 중간 상황 출력
        plt.figure(figsize=(20, 4))
        plt.suptitle('iter ' +str(i))
        for k in range(20):
            plt.subplot(2,10,k+1)
            img=generator.predict(np.random.normal(0,1,(1,noise_dimension)))
            plt.imshow(img[0].reshape(28,28),cmap='gray')
            plt.xticks([]); plt.yticks([])
        plt.show()
```



iteration : 1000



iteration : 2000



iteration : 3000



iteration : 4000

각 실습문제에 대해 code(+주석), 그래프, 분석 내용 등은 필수적으로 포함시킬 것

실습 #1) GAN을 이용한 MNIST 데이터 생성

- Functional API를 사용하여 설계
- MNIST 데이터에 대해 GAN 학습 진행
- GAN이 해결해야 될 문제 분석

실습 #2) 구조 변경 및 파라미터 조정에 따른 개선 및 비교분석

- Functional API를 사용하여 설계
 - MNIST 데이터에 대해 GAN 학습 진행
 - 다양한 방법으로 계층 변경하며 모델 개선
 - 구조 개선을 통한 MNIST 이미지 생성
- ex) Convolutional layer, Batch_normalization 추가 및 구조 변경 등등
- AE vs VAE vs GAN의 결과 비교분석 및 모델 구조에 따른 결과의 차이 분석